

Project Report - NourishNet Data Challenge

NourishNet Bot: Closing the Coverage Gap in Food Assistance Search

Github: <https://github.com/AmeyHengle/nafsi-NourishNet>

1. Background and Motivation

Picture a Wednesday evening in Anacostia. A church volunteer opens WhatsApp and types a quick message into the community group: "This Saturday 10am, free groceries at the church. Tell your neighbors." Within a few hours, the message has been forwarded across a dozen other groups. By Saturday morning, over 150 families have shown up, collected food, and gone home.

Across town, a single mother is searching FoodFinder on her phone that same Saturday morning. The nearest result is eight miles away. She does not own a car. She stays home. She had no idea that a church two blocks from her apartment had 150 boxes of free food that morning, because that event was never listed on any website. It existed only in a WhatsApp chat she was not part of.

This gap is not an edge case. It plays out every week across Washington DC and Maryland, in every neighbourhood where food events are organised informally and community members share information through the channels they already use. The food exists. The people who need it exist. What fails consistently is the connection between them.

We entered this challenge believing that the dominant reason for this failure is not a lack of effort from organisers, but a fundamental mismatch between how food event information is shared and how people search for it. Our submission, **NourishNet Bot**, is an automated system that closes that gap by reaching into the informal channels where food event information actually lives.

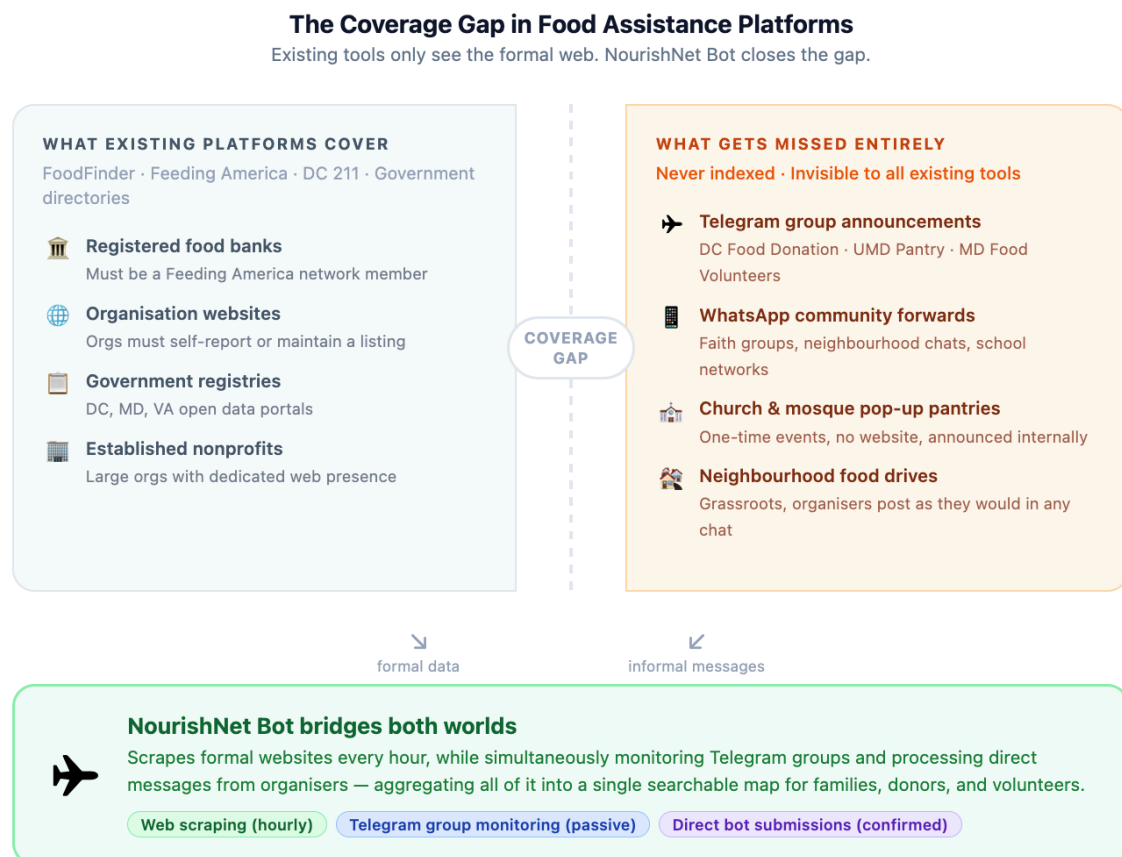
2. Why Food Event Search Has a Coverage Problem

2.1 What existing platforms can and cannot see

Existing tools in this space, including FoodFinder, the Feeding America food bank locator, DC's 211 helpline, and local government resource directories, share a common architecture. They maintain a curated database of registered, formal organisations: large

food banks, established nonprofits, government-run pantries. These tools work well for what they cover, and we do not seek to replace them.

The critical limitation is what they do not cover. All of these platforms rely on formal data sources: organisational websites, government registries, and self-reported listings. To appear on FoodFinder, an organisation must register. To appear in the Feeding America locator, a food bank must be a network member. The tools have good depth within the formal sector, but their coverage stops at the boundary of the formal web.



The diagram contrasts what existing platforms index (registered food banks, government data, established nonprofits) with what they miss entirely (Telegram groups, WhatsApp forwards, church pop-ups, neighbourhood drives), with NourishNet Bot shown as the bridge between both worlds.

2.2 Where the information actually lives

The reality on the ground is different. A significant share of food distribution activity, particularly the smallest, most local, and most time-sensitive events, never makes it onto a website. It circulates in Telegram groups, WhatsApp chats, Instagram stories, and email chains. A church pastor sends a message to a 200-person community group on a Tuesday afternoon: "Free groceries this Saturday, 10am, bring a bag." A neighbourhood

organiser forwards a flyer to three different chats. A community fridge gets restocked and someone posts about it. These events are real, they serve real people, and they are entirely invisible to every existing food search platform.

For families experiencing food insecurity, this is not a minor inconvenience. These informal events are often the most accessible: they happen in the neighbourhood, require no documentation, and serve communities that may not trust formal institutions. Missing them is not a data gap. It is a coverage failure.

2.3 Why solving this manually is not feasible

The obvious response is: why not hire people to monitor these channels and add events manually? We examined this question carefully, and the numbers make it impractical at any meaningful scale.

The DC and Maryland area has hundreds of active Telegram and WhatsApp groups related to food assistance, community support, faith communities, and neighbourhood organisations. A conservative estimate of just 50 relevant groups, each generating an average of 30 messages per day, produces 1,500 messages daily that would need to be read, evaluated, and acted upon. That is approximately four hours of uninterrupted reading every day by a single dedicated person, and that is before any data entry work begins. It also assumes that person is already a member of every relevant group, which is practically impossible given that many groups are private and membership-based.

Even a dedicated team of three full-time staff could not maintain consistent, real-time coverage across the informal information ecosystem of a metropolitan area. Events would still be missed. Coverage would still be incomplete. The human effort required scales linearly with the number of groups, while the number of groups continues to grow.

This is the problem that we are trying to solve using the NourishNet Bot.

3. Our Solution

Our submission consists of two tightly coupled components: **NourishNet Bot** and the **NourishNet Web App**. The bot is the core innovation, the mechanism through which we achieve broader coverage than existing platforms. The web app is the delivery layer, the interface through which families, donors, and volunteers access what the bot has collected.

3.1 NourishNet Bot

NourishNet Bot ([@nourishnet_bot](#) on Telegram) is an automated AI agent that collects food event information from two distinct channels simultaneously.

Direct submissions. Community organisers can message the bot directly with a plain text description of their event, or send a photo of an event flyer. The bot uses a large language model (Groq's Llama 3.1 70B) to extract structured details (name, address, date, hours, eligibility, food types, and contact) and sends the organiser a summary to confirm. If any required field is missing, the bot asks for it conversationally before queueing the submission. Once confirmed, the event is live on the web app within one hour.

Group monitoring. The bot can be added to any Telegram group where food events are discussed. It silently reads every message without disrupting the conversation. When it detects a message that describes a food event (identified through keyword detection and AI extraction), it adds it to the database automatically. The only visible action is a 👍 reaction on the message. The organiser does not need to know the bot exists, much less interact with it.

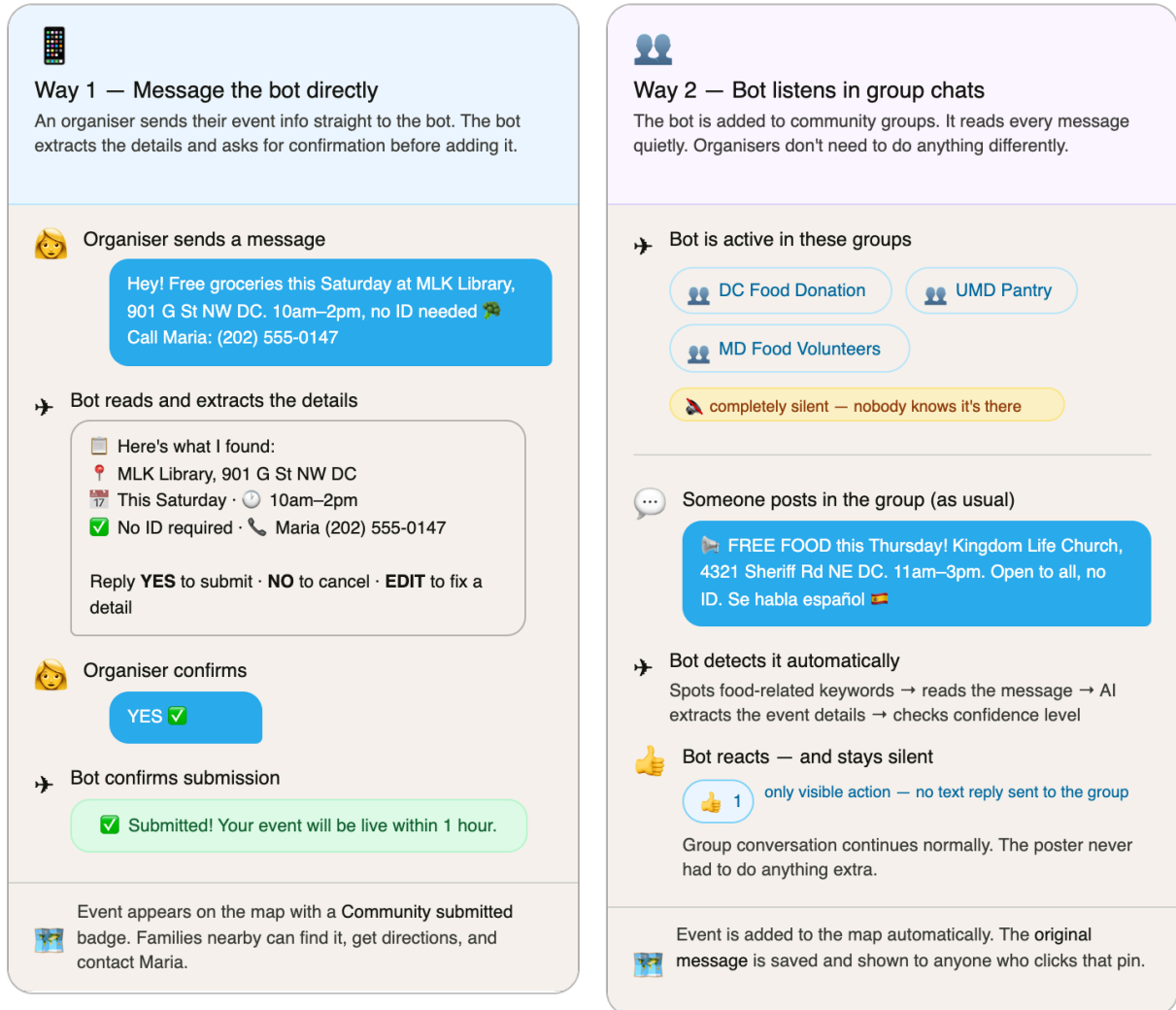
We currently have the bot active in three Telegram groups: **DC Food Donation**, **UMD Pantry**, and **Maryland Food Volunteers**. A single organiser posting their usual announcement to their usual group automatically contributes to the NourishNet database without any change to their behaviour.

This is the mechanism through which NourishNet Bot achieves broader coverage than existing platforms. We are not asking organisers to adopt a new tool. We are finding them where they already are.



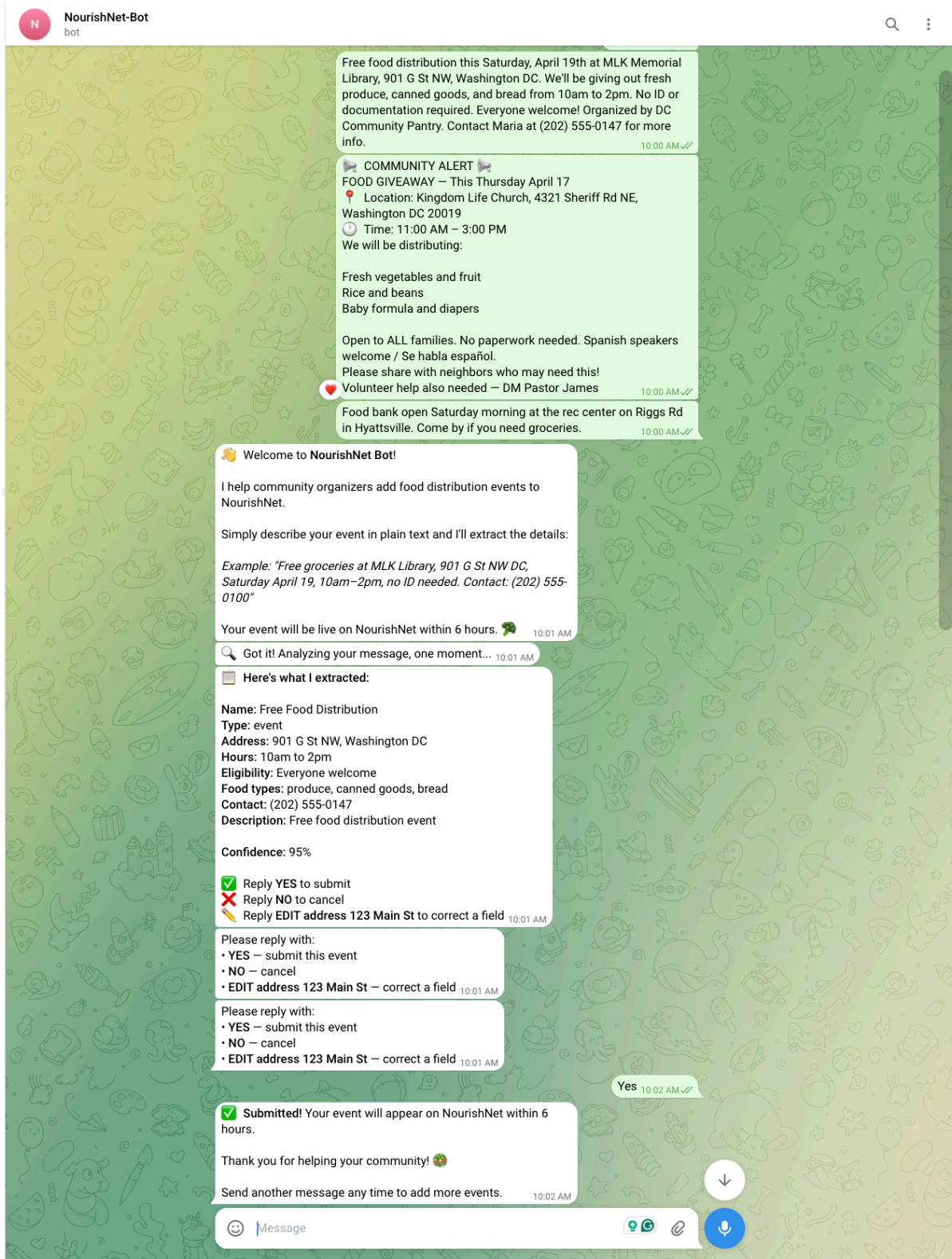
NourishNet Bot

@nourishnet_bot · two ways it collects food event information



The two-panel diagram shows the direct message flow (left) and the silent group monitoring flow (right), including the chat interface, the 🍌 reaction, and the outcome for the map.

The Telegram Bot in Action:

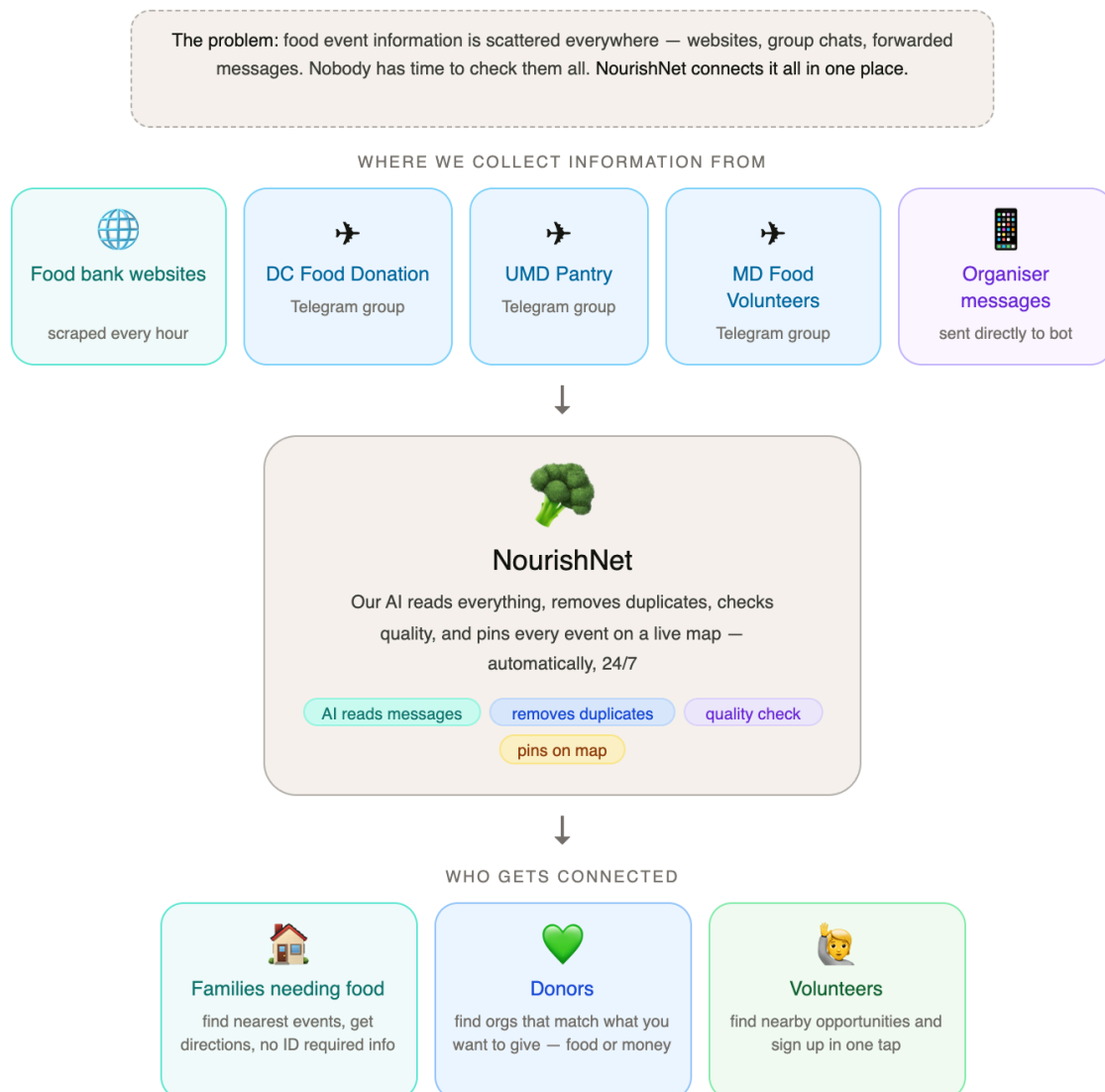


3.2 NourishNet Web App

The NourishNet Web App is a React application deployed at <https://ameyhengle.github.io/nafsi-NourishNet/>. It serves as the unified discovery interface for all information collected by the bot and the web scraping pipeline. It is role-

based, meaning families, donors, and volunteers each get a tailored experience, and it is fully bilingual in English and Spanish.

The web app is entirely static. It reads from a single `data.json` file that the pipeline writes to and commits back to the repository on every run. There is no backend server. GitHub Actions handles all automated execution.



The system architecture diagram shows all three input channels (web scraping, Telegram DMs, group monitoring) converging through the pipeline into `data.json`, which the React app reads from.

4. Intended Users

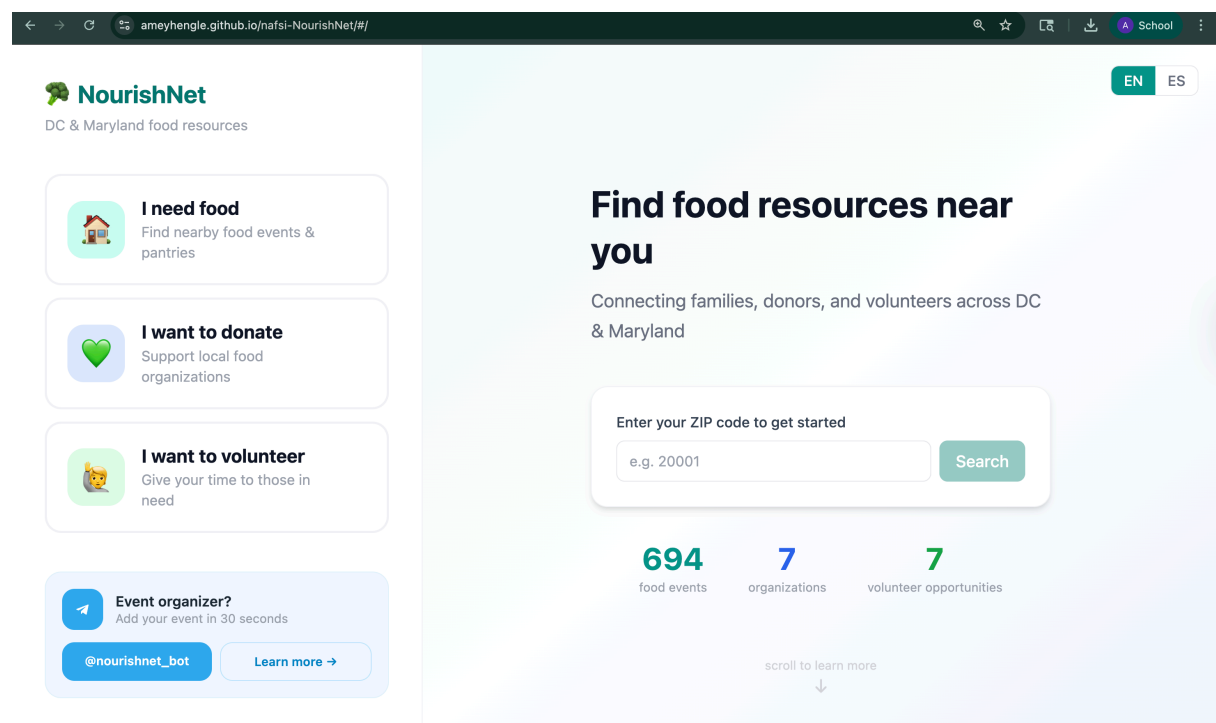
Families and households seeking food assistance are our primary user group. For these users, the most important pieces of information are proximity (where is the

nearest event?), timing (is it happening this week?), and access (do I need an ID? Is it open to everyone?). We designed the interface around these three questions specifically, with a ZIP-code-first search that immediately shows the nearest events on a map, sorted by distance. Every event card surfaces eligibility information and hours prominently, and provides a one-tap link to directions.

Donors, both individuals and organisations who want to contribute food or funds, need to understand which organisations are active near them, what items are most needed, and how to make contact. Our Donors view opens with a short preference quiz (food donations, monetary donations, or both) that filters and ranks organisations by relevance. Every organisation listing includes a "currently needs" section drawn directly from the source website or submitted description.

Volunteers looking to give their time need schedule information, a sense of what skills are required, and a clear path to sign up. Our Volunteers view surfaces opportunities sorted by proximity, with an urgency indicator for listings where spots are limited.

5. The User Interface



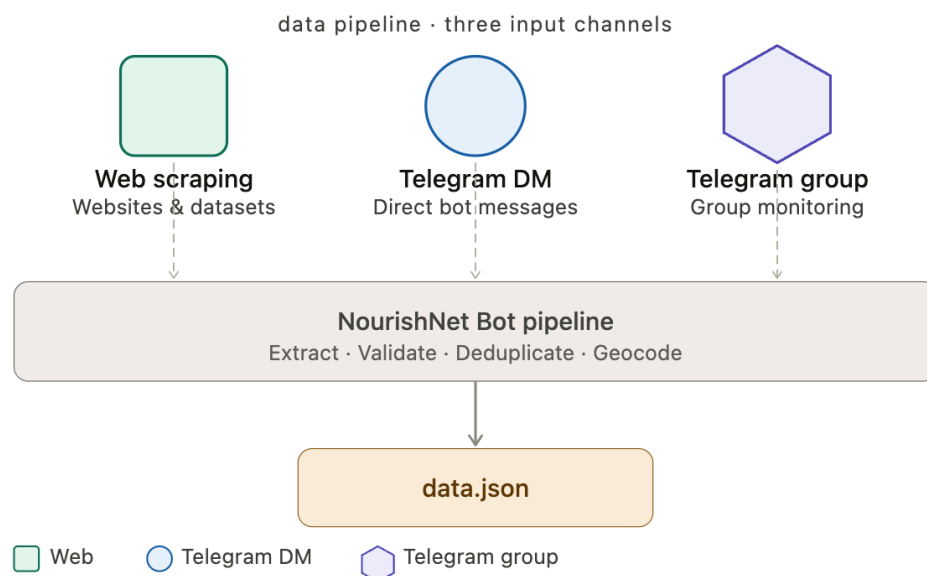
The NourishNet Web App opens with a two-column landing page: role-selection cards on the left, a ZIP code search on the right. Each role routes to a split-panel view, with a scrollable card list on one side and an interactive map on the other. The map uses a two-dimensional colour encoding: dot fill colour encodes entity type (teal for food events, blue for donor orgs, green for volunteer opportunities), and border colour encodes data source (white for web-scraped, Telegram blue for community-submitted, amber for demo data). Cards submitted via Telegram display the original message text and group

name when expanded, giving users a direct window into where the information came from.

A full step-by-step walkthrough for each user type is available on the live app at <https://ameyhengle.github.io/nafsi-NourishNet/#/how-it-works>.

6. The Data Pipeline: Three Channels, One Source of Truth

Our data pipeline operates across three parallel channels, all feeding into a single `data.json` file that the web app reads from.



6.1 Web scraping

An automated pipeline runs every hour via GitHub Actions. It reads from a growing directory of URLs, seeded with the provided dataset and expanded automatically over time, fetches each page, and sends the cleaned text to the Groq LLM for structured extraction. Every extracted record receives a confidence score. Records below a minimum threshold are discarded. Records between thresholds are flagged as unverified and shown with a warning badge in the UI. Only high-confidence records are published as active listings.

The pipeline also handles two data quality problems that naive scraping ignores. First, staleness: events with specific past dates are automatically expired and removed. Second, deduplication: the same food bank appearing on five different websites produces a single merged listing, using fuzzy name matching and geographic proximity checks.

6.2 NourishNet Bot: Direct submissions

Community organisers message `@nourishnet_bot` directly with a plain-text description of their event. The bot extracts the key details, presents a structured summary, and asks

the organiser to confirm or correct before submitting. If required fields are missing, most commonly the address or event date, the bot asks for them conversationally. The entire interaction typically takes under two minutes. Once confirmed, the event is live within one hour.

6.3 NourishNet Bot: Group monitoring

The bot silently monitors Telegram groups where food events are discussed. When it detects a relevant message (identified through keyword pre-filtering and AI extraction), it queues the event automatically if the confidence score meets our threshold. The only visible action is a 👍 reaction on the message. We currently monitor DC Food Donation, UMD Pantry, and Maryland Food Volunteers.

A three-layer deduplication system prevents the same event from appearing multiple times when an organiser posts in a group and also messages the bot directly: MD5 hashing of message text catches exact reposts, Groq confidence gating filters low-quality extractions, and sender-plus-time-window deduplication catches the same organiser across different submission paths.

6.4 Data schema

We defined a three-tier schema across the three entity types: food events, donor organisations, and volunteer opportunities. Each entity has required fields (without which it is not added to the database), recommended fields (which the bot prompts for if missing), and optional fields (accepted when present).

required	What's new	entity required without it	recommended	Not added if missing from submission
optional	Accepted if present, never deleted on			
Food Event entity_type: "event" → shown to families				
id	string	required		WHY IT MATTERS
name	string	required		Card title and map popup label
address	string	required		Decoded to lat/long — no address = no map pin
lat / long	number	required		Derived from address — geocoding must succeed
date_or_schedule	string	required		User needs to know when — without this the listing is useless
hours	string	recommended		Operations menu — needed for "open now" filter
eligibility	string	recommended		Do I qualify? Read 100 times prominently on card
food_types	string[]	recommended		Chip display + food type filter is UI
languages_served	string[]	recommended		ENES badge and language filter
contact	string	recommended		User may need to confirm or ask questions
contact_phone	string	optional		Reduces friction — critical for car-free families
organizer	string	optional		Name of person or org running the event
source_url	string	optional		"View details" button on card
Organization entity_type: "org" → shown to donors				
id	string	required		WHY IT MATTERS
name	string	required		Card title and map popup label
address	string	required		Donor needs to know where to drop off
lat / long	number	required		Map display and distance sorting
contact	string	required		Donor must reach the org before donating — hard requirement
description_types	string	recommended		Food, money, cash — shows donor self-identifying
hours	string	recommended		Drop-off hours — donor needs to plan around this
current_needs	string[]	recommended		Most needed items — motivates targeted donations
languages_served	string[]	recommended		Helps match bilingual donors to bilingual orgs
website	string	optional		"Get involved" and "Donate now" buttons on card
description	string	optional		1-2 paragraphs org overview shown on card
Volunteer Opportunity entity_type: "opportunity" → shown to volunteers				
id	string	required		WHY IT MATTERS
name	string	required		Show opportunity title on card
address	string	required		Where to show up — without this a volunteer can't come
lat / long	number	required		Map display and distance filter
contact	string	required		How to sign up — without this the listing is dead on arrival
hours	string	recommended		When to show up — volunteer must plan their schedule
contact_email	string	recommended		One-way messaging, feedback — key for volunteers
skills_needed	string[]	recommended		Cooking, lifting, lifting — helps self-selection
urgency_level	number	recommended		Urgency signal — "30 spots left" drives faster sign-up
minimum_age	number	optional		Important for families wanting to volunteer together
description	string	optional		What volunteers actually do

The schema reference table shows required, recommended, and optional fields for each entity type.

For food events, required fields are name, geocodable address, and date or schedule. For donor organisations: name, address, and a contact method. For volunteer

opportunities: name, address, and contact. This minimum bar ensures that every listing in the database is actionable. A user can always find it on the map and always has a way to reach the organiser.

7. Additional Data Sources

Beyond the seed CSV provided in the challenge, we incorporated the following additional sources into our URL directory: the DC Department of Human Services food assistance page, the Capital Area Food Bank event listings, the Maryland Food Bank partner network, Bread for the City, Martha's Table, SOME (So Others Might Eat), Manna Food Center, and the Jewish Social Service Agency food pantry. We also seeded mock data for demonstration purposes, clearly tagged and visually distinguished from live scraped data in the interface.

The pipeline's URL discovery module adds new sources automatically on each run by scoring outbound links found on already-scraped pages, so the dataset grows without any manual intervention.

8. Future Work

1. Our current implementation covers Telegram because its Bot API is open and requires no platform approval. WhatsApp's Business API requires Meta verification, which is beyond a hackathon timeline. Extending the same passive monitoring capability to WhatsApp groups would significantly expand coverage.
 2. The bot currently polls every 15 minutes via GitHub Actions. Moving it to a persistent lightweight server would make the interaction feel immediate. A community organiser would receive a reply within seconds rather than minutes.
 3. Many food events are announced as image flyers. Adding vision model support would allow the bot to read these directly, eliminating the need for organisers to retype information from a graphic they already have.
-

9. Conclusion

The food assistance information problem is not primarily a technology problem. It is a coverage problem, rooted in the fact that the most local, most accessible, and most time-sensitive food events happen in channels that no existing platform is designed to see.

NourishNet Bot's contribution is specifically to coverage. We built an automated agent that reads the informal web: the group chats, the forwarded messages, the community announcements, and connects that information to the people who need it, without requiring organisers to change how they communicate or adopt a new platform.

We are under no illusion that this solves food insecurity. What it does is reduce one specific, fixable source of friction: the gap between information that exists and information that reaches the people who need it.
